

Project 2 on Machine Learning

Classification and regression: writing our own neural network code

Deadline November 6 (midnight), 2026

Data Analysis and Machine Learning FYS-STK3155/FYS4155
Department of Physics, University of Oslo, Norway

October 6, 2026

Deliverables

First, join a group in Canvas with your group partners. Pick an available group for Project 2 in the **People** page.

In Canvas, deliver as a group and include:

- A PDF of your report which follows the guidelines covered below, in the exercises of week 39 and in the Tuesday session of week 40. Additional requirements include:
 - It should be around 5000 words; use the word counter in Overleaf for this. This often corresponds to 10–12 pages. References and appendices are excluded from the word count. Appendices should read like regular text and can include additional calculations you have made.
 - It should include around 10–15 figures. You can include more figures in appendices and/or as supplemental material in your repository. Including additional figures in appendices requires also that you add text which explains the additional material.
- A comment linking to your GitHub repository (or a folder in one of your GitHub repositories) for this project. The repository must include
 - a PDF file of the report;
 - a folder named `Code`, where you put Python files for your functions and notebooks for reproducing your results. Remember to use a seed for generating random data and for train-test splits when generating final results;
 - a `README` file with the names of the group members, a short description of the project, a description of how to install the required packages to run your code (from a `requirements.txt` file or similar, such as a plain text description), and the names and descriptions of the various notebooks in the `Code` folder and the results they produce.

Preamble: note on writing reports, using reference material, AI and other tools

We want you to answer the three different projects by handing in reports written like a standard scientific/technical report. The links at <https://github.com/EducationalMaterialUiO/MachineLearningUiO/tree/main/doc/Projects> contain more information. There you can find examples of previous reports, the projects themselves, how we grade reports etc. How to write reports will also be discussed during the various lab sessions. Please do ask us if you are in doubt. In that folder you will find how we evaluate the projects. You will receive a feedback

with a final score for each project.

When using codes and material from other sources, you should refer to these in the bibliography of your report, indicating wherefrom you for example got the code, whether this is from the lecture notes, software like **Scikit-Learn**, **JAX**, **TensorFlow**, **PyTorch** or other sources. These sources should always be cited correctly. How to cite some of the libraries is often indicated from their corresponding GitHub sites or websites; see for example how to cite Scikit-Learn at <https://scikit-learn.org/dev/about.html>.

You may use Large Language models (LLMs) like [ChatGPT](#), [Claude](#) or similar as a *supportive measure* when writing the report and developing your code, that is, as a tool that helps you understand, check and improve work which is your own, and not as a substitute for doing the analysis, writing the code or drawing the conclusions yourself. In the report, you should after the conclusions, write a declaration on how you have used LLMs following the instructions at https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/LLM_Use_Declaration_Guidelines.md. Failing to do so, leads to an automatic deduction of five (5) points out of the final score of 100 for the project. Remember that *you* are responsible for every line of code and every claim in the report: be prepared to explain and defend all of it. If you use LLMs, please take a look at the guidelines at <https://github.com/EducationalMaterialUiO/MachineLearningUiO/tree/main/doc/UsingLLMs> on how to use these tools efficiently.

If you would like to study other data sets, feel free to propose other sets. What we have proposed here are mere suggestions from our side. If you opt for another data set, consider using a set which has been studied in the scientific literature. This makes it easier for you to compare and analyze your results. Comparing with existing results from the scientific literature is also an essential element of the scientific discussion. The University of California at Irvine with its Machine Learning repository at <https://archive.ics.uci.edu/ml/index.php> is an excellent site to look up for examples and inspiration. [Kaggle.com](#) is an equally interesting site. Feel free to explore these sites and/or ask us for inputs relevant to your discipline.

Classification and regression, writing our own neural network code

The main aim of this project is to study both classification and regression problems by developing our own feed-forward neural network (FFNN) code. The lecture material and the exercises of weeks 41 and 42 (see <https://github.com/EducationalMaterialUiO/MachineLearningUiO/tree/main/doc/WeeklyMaterial/week41> and <https://github.com/EducationalMaterialUiO/MachineLearningUiO/tree/main/doc/WeeklyMaterial/week42>), together with Chapter 8 of the lecture notes (see https://educationalmaterialuio.github.io/MachineLearningUiO/doc/LectureNotes/_build/html/chapter8.html), should contain enough information for you to get started with writing your own code.

We will also reuse our codes on gradient descent methods from project 1.

The data sets that we propose here are (the default sets)

- **Regression** (fitting a continuous function). In this part you will need to bring back your results from project 1 and compare these with what you get from the neural network code to be developed here. The data sets could be
 - the simple one-dimensional Runge function from project 1, that is $f(x) = \frac{1}{1+25x^2}$. We recommend using a simpler function when developing your neural network code for regression problems. Feel however free to discuss and study other functions, such as the two-dimensional Runge function $f(x, y) = [(10x - 5)^2 + (10y - 5)^2 + 1]^{-1}$, or even more complicated two-dimensional functions (see the supplementary material of

<https://www.nature.com/articles/s41467-025-61362-4> for an extensive list of two-dimensional functions).

- **Classification.** We will consider a multiclass classification problem given by the full MNIST data set of handwritten digits, see <https://www.tensorflow.org/datasets/catalog/mnist> for a description of the data set and how to cite it.

The project is organized so that it follows the lectures and exercises of weeks 41 and 42 and Chapter 8 of the lecture notes:

- Part a): the cost functions, their gradients and the activation functions we need (Sections 8.6 and 8.7, and Sections 5.3 and 5.6 of Chapter 5 for the cross entropy and the softmax function).
- Part b): the feed-forward pass and the back propagation algorithm, and the first comparison with OLS from project 1 (Sections 8.5, 8.8, 8.10 and 8.13; weeks 40 and 41).
- Parts c) and d): activation functions, depth and width, and the L_1 and L_2 penalties (Sections 8.6, 8.9 and 8.12; week 42).
- Part e): classification with the softmax output layer and the cross entropy (Sections 8.11 and 8.14, and Section 5.12 on how to measure the quality of a classifier; weeks 41 and 42).
- Part f): the critical evaluation (Section 8.15).

The analytical parts (part a) belong in the theory section of your report. The project-style exercise at the end of Chapter 8 (Section 8.18) follows the same steps as this project and can be used as a checklist.

What you should take away from this project

Beyond the figures and tables in your report, these are the insights we want you to be able to state, explain and defend after working through the project. Use them as a checklist when you write your conclusions.

1. **A neural network replaces hand-made features by learned ones.** In project 1 the design matrix contained powers of x that you chose. A network takes the raw input and learns its own features in the hidden layers; the universal approximation theorem (Section 8.4) tells you that this is possible, not that it is easy.
2. **Backpropagation is reverse-mode automatic differentiation applied to a layered function.** The four equations of Section 8.8.4 are nothing but the chain rule organized so that one forward and one backward sweep give the gradient with respect to every weight and bias, at a cost of roughly two forward passes, independently of the number of parameters. You should be able to derive them and to check your implementation against `jax.grad`.
3. **The output activation and the cost function come in matched pairs.** Linear output with the MSE, sigmoid with the binary cross entropy and softmax with the multiclass cross entropy all give the same output error $\delta^L = \mathbf{a}^L - \mathbf{y}$ (up to the normalization). Mixing the pairs (for instance a sigmoid output with the MSE) gives tiny gradients exactly where the network is confidently wrong.
4. **The optimizer does not care where the gradient comes from.** Plain gradient descent, stochastic gradient descent, RMSprop and Adam from project 1 carry over unchanged; what changes is that the cost function is no longer convex, so the learning rate, the initialization and the minibatch size now decide whether you converge at all, and to which minimum.
5. **Initialization and scaling are not details.** Unscaled inputs or weights of the wrong size saturate the sigmoid, and the gradient vanishes (Sections 8.9 and 8.10). Standardized inputs, Xavier or He initialization and the ReLU family exist precisely to keep the gradients

alive.

6. **Hyperparameters must be searched systematically, and on validation data.** The learning rate γ , the penalty λ , the depth, the width and the activation function interact. A grid of two parameters at a time, shown as a heat map, is far more informative than a handful of hand-picked runs, and the test set is used only once, at the end.
7. **Overfitting looks the same as in project 1.** A growing gap between training and test error as the number of parameters grows is the same bias-variance story as for the polynomial degree, and the L_1 and L_2 penalties play the same role as in Ridge and Lasso regression; early stopping is a form of regularization that costs nothing.
8. **More flexible is not automatically better.** For a smooth one-dimensional function, OLS with a well-chosen polynomial degree is a closed-form, cheap and hard-to-beat benchmark. The neural network earns its cost only when the features are not known in advance, as for the pixels of MNIST. Your report should say when each method is the right tool.
9. **A number is meaningless without a baseline.** An accuracy of 97% on MNIST is good only if you know that guessing gives 10%, that logistic regression on raw pixels gives roughly 92% and that convolutional networks reach above 99%. A confusion matrix tells you which errors remain; the accuracy alone does not.
10. **A code you cannot verify is a code you cannot trust.** Gradient checks against automatic differentiation, recovering the closed-form OLS solution with a linear network, and comparing with Scikit-Learn or PyTorch are the tests that turn your implementation into a result you can defend. Seeds and documented hyperparameters make the result reproducible.

We will start with a regression problem and we will reuse our codes on gradient descent methods from project 1.

Part a): Analytical warm-up

When using our gradient machinery from project 1, we will need the expressions for the cost/loss functions and their respective gradients. The functions whose gradients we need are:

1. the mean-squared error (MSE) with and without the L_1 and L_2 norms (regression problems);
2. the binary cross entropy (aka log loss) for binary classification problems with and without the L_1 and L_2 norms;
3. the multiclass cross entropy cost/loss function (aka softmax cross entropy or just softmax loss function).

Set up these three cost/loss functions and their respective derivatives and explain the various terms. In this project you will however only use the MSE and the softmax cross entropy. The cross entropy and the softmax function are discussed in Sections 5.3 and 5.6 of the lecture notes, the cost functions for neural networks in Section 8.7, and the regularization terms in Section 8.12.

We will test three activation functions for our neural network setup, these are

1. the Sigmoid (aka *logit*) function,
2. the RELU function and
3. the Leaky RELU function.

Set up their expressions and their first derivatives. The activation functions are discussed in Section 8.6 of the lecture notes (the sigmoid in Section 8.6.1, the RELU family in Section 8.6.2), and in the lecture material of week 41 at <https://github.com/EducationalMaterialUiO/MachineLearningUi0/tree/main/doc/WeeklyMaterial/week41>.

Reminder about the gradient machinery from project 1

In the setup of a neural network code you will need your gradient descent codes from project 1. For neural networks we will recommend using stochastic gradient descent (Section 4.7 of the lecture notes) with either the RMSprop or the Adam algorithms for updating the learning rates (Sections 4.10 and 4.11). But you should feel free to try plain gradient descent as well. As in project 1 and in the lecture notes we call the learning rate γ (many texts call it η).

We recommend reading chapter 8 on optimization from the textbook of Goodfellow, Bengio and Courville at <https://www.deeplearningbook.org/>. This chapter contains many useful insights and discussions on the optimization part of machine learning. A useful reference on the back propagation algorithm is Nielsen's book at <http://neuralnetworksanddeeplearning.com/>.

You will find the Python [Seaborn package](#) useful when plotting the results as function of the learning rate γ and the hyperparameter λ .

Part b): Writing your own neural network code

Your aim now, and this is the central part of this project, is to write your own FFNN code implementing the back propagation algorithm discussed in the lecture material of week 41 (see <https://github.com/EducationalMaterialUiO/MachineLearningUiO/tree/main/doc/WeeklyMaterial/week41>) and week 42 (see <https://github.com/EducationalMaterialUiO/MachineLearningUiO/tree/main/doc/WeeklyMaterial/week42>), and in Section 8.8 of the lecture notes. The notation and the feed-forward pass are set up in Section 8.5, the general dense-layer case and the four equations of backpropagation in Sections 8.8.3 and 8.8.4, and the full algorithm in Section 8.8.5. A complete network from scratch, including a check of the gradient against automatic differentiation, is discussed in Section 8.13. The exercises of week 41 ([exerciseweek41.ipynb](#)) give you a template for the code: samples are stored in the rows of the input, the weight matrix of a layer has shape (inputs, outputs), and the feed-forward step of a layer reads $\mathbf{a} @ \mathbf{W} + \mathbf{b}$.

We will focus on a regression problem first, using the one-dimensional Runge function

$$f(x) = \frac{1}{1 + 25x^2},$$

from project 1.

Use only the mean-squared error as cost function (no regularization terms) and write an FFNN code for a regression problem with a flexible number of hidden layers and nodes using only the Sigmoid function as activation function for the hidden layers. Initialize the weights using a normal distribution (see Section 8.10 for how the width of that distribution should depend on the number of inputs of a layer). How would you initialize the biases? And which activation function would you select for the final output layer? And how would you set up your design/feature matrix? Hint: does it have to represent a polynomial approximation as you did in project 1?

Train your network and compare the results with those from your OLS regression code from project 1 using the one-dimensional Runge function. When comparing your neural network code with the OLS results from project 1, use the same data sets which gave you the best MSE score. Moreover, use the polynomial order from project 1 that gave you the best result. Compare these results with your neural network with one and two hidden layers using 50 and 100 hidden nodes, respectively.

Comment your results and give a critical discussion of the results obtained with the OLS code from project 1 and your own neural network code. Make an analysis of the learning rates employed to find the optimal MSE score. Test both stochastic gradient descent with RMSprop and Adam and plain gradient descent with different learning rates.

You should, as you did in project 1, scale your data (Section 3.13 of the lecture notes).

Part c): Testing different activation functions and depths of the neural network

You should also test different activation functions for the hidden layers. Try out the Sigmoid, the RELU and the Leaky RELU functions and discuss your results (Section 8.6.4 of the lecture notes discusses which activation to use, and Section 8.9 why the sigmoid makes the gradients vanish in deep networks). Test your results as functions of the number of hidden layers and nodes. Do you see signs of overfitting? It is optional in this project to perform a bias-variance trade-off analysis.

Part d): Testing different norms

Finally, still using the one-dimensional Runge function, add now the hyperparameters λ with the L_2 and L_1 norms (Section 8.12 of the lecture notes). Find the optimal results for the hyperparameters λ and the learning rates γ and neural network architecture and compare the L_2 results with Ridge regression from project 1 and the L_1 results with the Lasso calculations of project 1. Use again the same data sets and the best results from project 1 in your comparisons.

Part e): Classification analysis using neural networks

With a well-written code it should now be easy to change the activation function for the output layer.

Here we will change the cost function for our neural network code developed in parts b)–d) in order to perform a classification analysis (Section 8.11 of the lecture notes). The classification problem we will study is the multiclass MNIST problem of handwritten digits (LeCun, Cortes and Burges), see the description of the data set at <https://www.tensorflow.org/datasets/catalog/mnist>. We will use the softmax cross entropy function discussed in a). In this project you should use the *full* MNIST data set (70 000 images of 28×28 pixels), the one used in the Monday lecture of week 41; the code of that lecture reads exactly the file described below. Note that the Tuesday session of week 41 (and Section 8.14 of the lecture notes) uses the much smaller digits data set of Scikit-Learn (`load_digits`, 1797 images of 8×8 pixels, a different data set of the same kind and not a subset of MNIST). The reduced set is convenient for developing and debugging your code, since every run takes seconds, but the results you report should be for the full MNIST set.

The data set comes with a canonical split into 60 000 training images and 10 000 test images, each a 28×28 array of grey-scale pixel values between 0 and 255. It can be downloaded, without registering anywhere, as a single NumPy archive (about 11 MB) from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz> and loaded with

```
import numpy as np
with np.load("mnist.npz") as data:
    x_train, y_train = data["x_train"], data["y_train"]
    x_test, y_test = data["x_test"], data["y_test"] # untouched until the end
print(x_train.shape, y_train.shape, x_test.shape, y_test.shape)
# (60000, 28, 28) (60000,) (10000, 28, 28) (10000,)
```

A feed-forward network takes a vector as input, so flatten each image to a vector of 784 features and scale the pixel values to the interval $[0, 1]$ (scaling makes a real difference for gradient descent, see Section 3.13 of the lecture notes),

```
x_train = x_train.reshape(len(x_train), -1) / 255.0
x_test = x_test.reshape(len(x_test), -1) / 255.0
```

Use the canonical split the way it is meant to be used: all training, model selection and tuning of hyperparameters (learning rate, penalty λ , number of layers and nodes, activation functions) is done on the training data only, for example by setting aside a validation set from the training data or by cross-validation (Chapter 3 of the lecture notes). The test set is used *once*, at the very end, to report the performance of the models you have selected. If you are limited by computing time, you may train on a subset of the training images, but evaluate on the full test set.

Remember to cite the data set in your report, as you would cite any other source; the citation is given on the data set page above (Y. LeCun, C. Cortes and C. J. C. Burges, *The MNIST database of handwritten digits*, <http://yann.lecun.com/exdb/mnist/>).

Feel free to suggest other data sets. If you find the classic MNIST data set somewhat limited, try the Fashion-MNIST data set of Zalando Research at <https://github.com/zalandoresearch/fashion-mnist>. It has exactly the same format and the same 60 000/10 000 split, with ten classes of clothing items instead of digits, and the download links, loading code and the reference to cite are all on that page.

To measure the performance of our classification problem we will use the so-called *accuracy* score (Section 5.12 of the lecture notes). The accuracy is as you would expect just the number of correctly guessed targets t_i divided by the total number of targets, that is

$$\text{Accuracy} = \frac{\sum_{i=1}^n I(t_i = y_i)}{n},$$

where I is the indicator function, 1 if $t_i = y_i$ and 0 otherwise. Here t_i represents the target and y_i the outputs of your FFNN code and n is simply the number of targets t_i .

Discuss your results and give a critical analysis of the various parameters, including hyperparameters like the learning rates and the regularization parameter λ , various activation functions, number of hidden layers and nodes and activation functions.

Again, we strongly recommend that you compare your own neural network code for classification and pertinent results against a similar code using **Scikit-Learn** or **TensorFlow/Keras** or **PyTorch** (see the paragraph on established libraries in Section 8.14 of the lecture notes).

You should also compare your neural network results with those from logistic regression (Sections 5.3 and 5.6 of the lecture notes). Logistic regression is the *baseline* for the classification part: it is the classifier with no hidden layers, and a network with one or more hidden layers is only worth its extra cost if you can show that it performs better than this baseline. Your neural network code implements the equivalent of logistic regression by simply setting the number of hidden layers to zero and keeping just the input and the output layers (with the softmax activation and the cross entropy cost from part a)).

In addition to the accuracy, you should compute and interpret the *confusion matrix* of your best classification model (Section 5.12 of the lecture notes, see also https://www.researchgate.net/figure/Confusion-matrix-of-MNIST-and-F-MNIST-embeddings_fig5_349758607). The confusion matrix shows which digits are confused with which, something a single accuracy number hides, and if you also report the precision, the recall and the F_1 score, the confusion matrix is what makes these numbers interpretable. Scikit-Learn's `confusion_matrix` and `ConfusionMatrixDisplay` from `sklearn.metrics` are convenient for this.

If you wish to compare with logistic regression from **Scikit-Learn** as well, the following code uses the flattened and scaled arrays from above

```
from sklearn.linear_model import LogisticRegression
# Initialize the model (multinomial logistic regression with the saga solver)
```

```

model = LogisticRegression(solver='saga', max_iter=1000, random_state=42)
# Train the model
model.fit(x_train, y_train)
from sklearn.metrics import accuracy_score
# Make predictions on the test set
y_pred = model.predict(x_test)
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.4f}")

```

Part f): Critical evaluation of the various algorithms

After all these glorious calculations, you should now summarize the various algorithms and come with a critical evaluation of their pros and cons. Which algorithm works best for the regression case and which is best for the classification case? Section 8.15 of the lecture notes gives a top-down view of what neural networks can and cannot do that you may find useful here. These codes can also be part of your final project 3, but now applied to other data sets.

Summary of methods to implement and analyze

Required implementation:

1. Reuse the regression code and results from project 1; these will act as a benchmark for seeing how suited a neural network is for this regression task.
2. Implement a neural network with
 - a flexible number of layers;
 - a flexible number of nodes in each layer;
 - a changeable activation function in each layer (Sigmoid, ReLU, Leaky ReLU, as well as linear and softmax);
 - a changeable cost function, which will be set to the MSE for regression and the cross entropy for multiclass classification;
 - an optional L_1 or L_2 norm of the weights and biases in the cost function (only used for computing gradients, not as interpretable metrics).
3. Implement the back propagation algorithm to compute the gradient of your neural network.
4. Reuse the implementation of plain and stochastic gradient descent from project 1 (and adapt the code to work with your neural network)
 - with no optimization algorithm;
 - with RMSprop;
 - with Adam.
5. Implement logistic regression as the baseline classification model (equivalent to a neural network with just the output layer, that is with zero hidden layers).
6. Implement scaling and train-test splitting of your data, preferably using Scikit-Learn (MNIST comes with its own test set; tune on the training data and use the test set only for the final evaluation).
7. Implement and compute metrics like the MSE, the accuracy and the confusion matrix.

Required analysis:

1. Briefly show and argue for the advantages and disadvantages of the methods from project 1.
2. Explore and show the impact of changing the number of layers, nodes per layer, choice of activation function, and inclusion of L_1 and L_2 norms. Present only the most interesting results from this exploration. 2D heat maps will be good for this: start with finding a well

- performing set of hyperparameters, then change two at a time in a range that shows good and bad performance.
3. Show and argue for the advantages and disadvantages of using a neural network for regression on your data.
 4. Show and argue for the advantages and disadvantages of using a neural network for classification on your data. In particular, show whether and why a network with hidden layers performs better than the logistic-regression baseline (no hidden layers).
 5. Compute and interpret a confusion matrix of your best classification model (Section 5.12 of the lecture notes and https://www.researchgate.net/figure/Confusion-matrix-of-f-MNIST-and-F-MNIST-embeddings_fig5_349758607).
 6. Show and argue for the advantages and disadvantages of the different gradient methods and learning rates when training the neural network.

Background literature

- The lecture notes of the course, in particular Chapter 8 (neural networks, at https://educationalmaterialuio.github.io/MachineLearningUiO/doc/LectureNotes/_build/html/chapter8.html), Chapter 4 (stochastic gradient descent, RMSprop, Adam and automatic differentiation) and Chapter 5 (the cross entropy, the softmax function and how to measure the quality of a classifier).
- The text of Michael Nielsen is highly recommended, see Nielsen's book at <http://neuralnetworksanddeeplearning.com/>. It is an excellent read.
- Goodfellow, Bengio and Courville, Deep Learning, at <https://www.deeplearningbook.org/>. Here we recommend chapters 6, 7 and 8.
- Raschka et al. at <https://sebastianraschka.com/blog/2022/ml-pytorch-book.html>. Here we recommend chapters 11, 12 and 13.

Introduction to numerical projects

Here follows a brief recipe and recommendation on how to answer the various questions when preparing your answers.

- Give a short description of the nature of the problem and the eventual numerical methods you have used.
- Describe the algorithm you have used and/or developed. Here you may find it convenient to use pseudocoding. In many cases you can describe the algorithm in the program itself.
- Include the source code of your program. Comment your program properly. You should have the code at your GitHub/GitLab link. You can also place the code in an appendix of your report.
- If possible, try to find analytic solutions, or known limits in order to test your program when developing the code. In this project the gradient from automatic differentiation and the closed-form OLS solution of project 1 are exactly such tests for your neural network code.
- Include your results either in figure form or in a table. Remember to label your results. All tables and figures should have relevant captions and labels on the axes.
- Try to evaluate the reliability and numerical stability/precision of your results. If possible, include a qualitative and/or quantitative discussion of the numerical stability, eventual loss of precision etc.
- Try to give an interpretation of your results in your answers to the problems.
- Critique: if possible include your comments and reflections about the exercise, whether you felt you learnt something, ideas for improvements and other thoughts you've made

when solving the exercise. We wish to keep this course at the interactive level and your comments can help us improve it.

- Try to establish a practice where you log your work at the computer lab. You may find such a logbook very handy at later stages in your work, especially when you don't properly remember what a previous test version of your program did. Here you could also record the time spent on solving the exercise, various algorithms you may have tested or other topics which you feel worthy of mentioning.

Format for electronic delivery of report and programs

The preferred format for the report is a PDF file. You can also use DOC or postscript formats or an IPython notebook file. As programming language we prefer that you choose between C/C++, Fortran2008, Julia or Python. The following prescription should be followed when preparing the report:

- Use Canvas to hand in your projects, log in at <https://www.uio.no/english/service/s/it/education/canvas/> with your normal UiO username and password.
- Upload **only** the report file or the link to your GitHub/GitLab or similar type of repository! For the source code file(s) you have developed please provide us with your link to your GitHub/GitLab or similar domain. The report file should include all of your discussions and a list of the codes you have developed. Do not include library files which are available at the course homepage, unless you have made specific changes to them.
- In your GitHub/GitLab or similar repository, please include a folder which contains selected results. These can be in the form of output from your code for a selected set of runs and input parameters.

Finally, we encourage you to collaborate. Optimal working groups consist of 2–3 students. You can then hand in a common report.

Software and needed installations

If you have Python installed (we recommend Python 3) and you feel pretty familiar with installing different packages, we recommend that you install the following Python packages via `pip` as

```
pip install numpy scipy matplotlib ipython jupyter scikit-learn
pip install pandas seaborn
pip install jax jaxlib
```

(`autograd` is a lighter alternative to JAX if you want to check your back propagation code against automatic differentiation; PyTorch or TensorFlow are needed only if you choose them for the comparison with an established library). For Python 3, replace `pip` with `pip3` where needed.

For OSX users we recommend also, after having installed Xcode, to install `brew`. Brew allows for a seamless installation of additional software via for example `brew install python3`. For Linux users, with its variety of distributions like for example the widely popular Ubuntu distribution, you can use `pip` as well and simply install Python as `sudo apt-get install python3` etc.

If you don't want to install various Python packages with their dependencies separately, we recommend [Anaconda](#), an open source distribution of the Python and R programming languages for large-scale data processing, predictive analytics, and scientific computing, that aims to simplify package management and deployment. Package versions are managed by the package management system `conda`.

Popular software packages written in Python for ML are [Scikit-learn](#), [JAX](#), [TensorFlow](#), [PyTorch](#) and [Keras](#). These are all freely available at their respective GitHub sites. They encompass communities of developers in the thousands or more, and the number of code developers and contributors keeps increasing.